

Directives

Extending HTML with your own behavior — a Directive is a class that can change the **appearance or behavior** of an HTML element, without being a whole new Component.

1 · THE CONCEPT

What is a directive?

In fact, a Component *is* a directive — just one with its own template. Sometimes we don't want a whole new Component — we just want to **add** a small behavior to an existing element (show/hide it, change its color, repeat it). Goal: reuse a small piece of DOM logic across many elements without duplicating code.

STRUCTURAL DIRECTIVES

Prefixed with `*` — add or remove elements from the DOM entirely

Examples: `*ngIf`, `*ngFor`, `@if`, `@for`

ATTRIBUTE DIRECTIVES

Change how an *existing* element looks or behaves, without touching the DOM tree

Examples: `ngClass`, `ngStyle`, a custom hover directive

2 · STRUCTURAL DIRECTIVES

Conditionally show or repeat elements

Modern (Angular 17+) control flow

```
@if (isLoggedIn) {  
  <p>Welcome back!</p>  
}  
  
@for (item of items; track item.id) {  
  <li>{{ item.name }}</li>  
}
```

Legacy syntax (still seen in older projects)

```
<p *ngIf="isLoggedIn">Welcome back!</p>  
<li *ngFor="let item of items">{{ item.name }}</li>
```

`@if` and `@for` are the modern, recommended way to conditionally show or repeat elements — they compile to more efficient code and don't need `CommonModule` imported. `*ngIf` / `*ngFor` are the older directive-based syntax — good to recognize in existing codebases. Unlike `display:none`, a false `*ngIf` takes the element out of the DOM completely.

3 · ATTRIBUTE DIRECTIVES (BUILT-IN)

ngClass & ngStyle — change how an element looks

```
<!-- toggle CSS classes based on conditions -->
<div [ngClass]="{ active: isActive, disabled: isDisabled }">
  Status box
</div>

<!-- apply inline styles dynamically -->
<div [ngStyle]="{ color: isError ? 'red' : 'green' }">
  Message
</div>
```

Both **ngClass** and **ngStyle** change how an element **looks**, without adding or removing it from the page — that's what makes them attribute directives.

4 · BUILDING YOUR OWN · APPHIGHLIGHT

A custom hover-highlight directive

Problem: what if none of the built-in directives do what you need — e.g. "highlight this element when the mouse hovers over it"? Write the shared behavior once as a Directive, then reuse it anywhere with a simple attribute — the same "don't repeat yourself" idea as Services, applied to DOM behavior instead of data logic.

1 · Generate

```
$ ng generate directive directives/highlight
```

2 · Write the logic

```
@Directive({ selector: '[appHighlight]' })
export class HighlightDirective {
  private el = inject(ElementRef);

  @HostListener('mouseenter')
  onMouseEnter() { this.el.nativeElement.style.backgroundColor = 'yellow'; }

  @HostListener('mouseleave')
  onMouseLeave() { this.el.nativeElement.style.backgroundColor = ''; }
}
```

3 · Use it anywhere

```
<p appHighlight>Hover over me!</p>
<div appHighlight class="card">This whole card highlights too.</div>
```

- **selector:** `'[appHighlight]'` — square brackets mean it's used as an attribute, e.g. `<p appHighlight>`.
- **ElementRef** gives direct access to the actual HTML element it's attached to.
- **@HostListener('mouseenter')** runs automatically when that browser event fires on the host element — no manual event binding needed in the template.

Stopping the wrong keystroke before it lands

Not every directive reacts to the mouse. This one listens on (`keydown`) and blocks a keystroke **before** it ever reaches the input — instead of letting a letter in and validating it afterwards. It also sanitizes pasted text, since paste doesn't fire `keydown` per character.

```
@Directive({ selector: '[appNumbersOnly]' })
export class NumbersOnlyDirective {
  readonly blocked = output<string>();

  private readonly allowedKeys = new Set([
    'Backspace', 'Delete', 'Tab', 'ArrowLeft', 'ArrowRight', 'Home', 'End',
  ]);

  @HostListener('keydown', ['$event'])
  onKeyDown(e: KeyboardEvent) {
    if (this.allowedKeys.has(e.key) || e.ctrlKey || e.metaKey) return;
    if (!/^[0-9]$/.test(e.key)) {
      e.preventDefault();
      this.blocked.emit(e.key);
    }
  }

  @HostListener('paste', ['$event'])
  onPaste(e: ClipboardEvent) {
    const text = e.clipboardData?.getData('text') ?? '';
    if (/^[^0-9]/.test(text)) e.preventDefault();
  }
}

<input appNumbersOnly (blocked)="onBlocked($event)">
```

Notice the allow-list of navigation keys — without it, `Backspace` or the arrow keys would get blocked too, since they aren't digits either. `output()` lets the directive tell its host component what happened, the same way a component's own `@Output()` would.

Real-world usage: phone number fields, OTP codes, quantity steppers, postal codes.

6 · BEST PRACTICES

When (and how) to write one

- Create a custom directive only when the **same** small behavior is needed on multiple, unrelated elements.
- Keep a directive focused on **one responsibility** — `appHighlight` just highlights, `appNumbersOnly` just filters keystrokes — don't turn it into a mini-component.
- Prefer built-in directives (`@if`, `@for`, `ngClass`) whenever they already solve the problem.
- Use `@HostListener` to react to events on the host element, and `@HostBinding` (or direct style/class writes) to change it — that pair covers most attribute directives.